

WITNESS-2 Preregistration

Length-Prefixed Quantum Nonce with Record-Hash Field Integrity

Series: WITNESS (second record)

Author: Derek Adam Hone / Remnant Fieldworks Inc.

Preregistration lock date: 2026-07-18 (UTC)

Status: PREREGISTERED — locked before execution

1. Claim (single, falsifiable)

A ProofRecord constructed with a length-prefixed quantum nonce and an independent `record_hash` field-integrity seal can be:

- (a) verified end-to-end against the provider's job record (W2-C1),
- (b) detected as tampered when any single field is substituted without recomputation (W2-C2),
- (c) rejected as a cross-context replay — by `record_hash` when the replayer does not recompute it, and by ARK-457 5-dimensional context binding when the replayer does (W2-C3) —

under the tested construction, with the design boundary (`record_hash` = field integrity only; `context_id` enforcement = separate ARK-457 layer) explicitly demonstrated.

2. Non-Claims (explicit)

- Does NOT prove quantum randomness superiority over CSPRNG. No Bell test; device bias present and acknowledged.
 - Does NOT prove universal security. Results apply only within the tested construction, circuit model, backend, qubits, calibration snapshot, shot count, and software harness.
 - Does NOT constitute a production certification or cryptographic security proof.
 - `record_hash` is a field-integrity seal, not a context-binding mechanism. An attacker who knows the schema can recompute `record_hash` after altering `context_id`. Context-id enforcement is deliberately delegated to the ARK-457 authorization layer. This design boundary is tested explicitly in W2-C3(b) and is not a defect.
-

3. Circuit Specification

- **Qubits:** 8
- **Circuit:** Hadamard on all 8 qubits, measure all — one circuit only.
- **Shots:** 4,000, one job submission.
- **Channel:** `ibm_quantum_platform`
- **Execution mode:** Batch

- **Backend selection:** Operational, non-simulator, ≥ 8 qubits, accessible on open plan. Least-busy at submission time. Backend name, calibration snapshot, timestamp, job ID, and provider instance (CRN) recorded in `results/raw/`.
- **QPU time budget gate:** `usage_remaining_seconds >= 30`. If insufficient time remains, GATE-STOP — do not purchase time, do not proceed.
- **Provider API unavailability at W2-C1 provider verification step:** GATE-STOP, not FAIL. Logged with reason.

4. Nonce Construction (length-prefixed)

```
LP(x) = struct.pack('>I', len(x)) + x    (4-byte big-endian uint32 length prefix)

quantum_nonce = SHA-256(
    LP(canonical_json(raw_counts).encode('utf-8'))
    || LP(job_id.encode('utf-8'))
    || LP(canonical_json(calibration_snapshot).encode('utf-8'))
)
```

- **Canonical JSON:** `json.dumps(obj, sort_keys=True, separators=(',', ':'), ensure_ascii=True)`
Output is a `str`; encoded to UTF-8 bytes before hashing.
- **LP prefix:** Prevents two distinct component splits from producing the same byte stream (length-extension ambiguity). Upgrade from WITNESS-1's bare concatenation.
- **Result:** 64-character lowercase hex string.

5. Calibration Snapshot (10-field deterministic schema)

All 10 fields always present. Missing values → `None` (not omitted). Non-finite floats (NaN, $\pm\text{Inf}$) → `None`. Keys in sub-dicts sorted lexicographically via `canonical_json`.

Field	Type	Notes
<code>backend_name</code>	str	Backend identifier
<code>backend_version</code>	str None	From configuration.backend_version
<code>last_update_utc</code>	str None	ISO 8601 second-precision UTC
<code>basis_gates</code>	[str]	Sorted lexicographically
<code>selected_qubits</code>	[int]	Sorted ascending
<code>readout_error_by_qubit</code>	{"<q>": float\ None}	By qubit index string
<code>t1_by_qubit</code>	{"<q>": float\ None}	Seconds
<code>t2_by_qubit</code>	{"<q>": float\ None}	Seconds
<code>gate_error_for_used_gates</code>	{"<gate>q<idx>": float\ None}	Gates: h, measure
<code>gate_length_for_used_gates</code>	{"<gate>q<idx>": float\ None}	Seconds

6. ProofRecord Schema (witness-proofrecord-1.0)

```
{
  "schema_version":      "witness-proofrecord-1.0",
  "quantum_nonce":      "<64-char hex, length-prefixed SHA-256>",
  "job_id":              "<IBM Quantum job ID string>",
  "backend":             "<backend name string>",
  "provider_instance":   "<CRN / provider instance identifier>",
  "calibration_hash":    "<SHA-256 hex of canonical-
al_json(calibration_snapshot)>",
  "raw_counts_hash":     "<SHA-256 hex of canonical_json(raw_counts)>",
  "context_id":          "witness-2-primary",
  "timestamp_utc":       "<ISO-8601 UTC string, second precision>",
  "record_hash":         "<SHA-256 hex of canonical_json(all fields above, sor-
ted)>"
}
```

record_hash construction:

```
record_hash = SHA-256(canonical_json({all fields except record_hash itself}))
```

- Provides field integrity: any single field altered without recomputing `record_hash` is detected immediately.

- Does NOT enforce `context_id` binding. An attacker may alter `context_id` and recompute a valid `record_hash`. Context-id enforcement requires ARK-457. See §7.

7. Preregistered Test Cases & Pass/Fail Criteria

W2-C1 — Honest End-to-End Verification

Procedure:

1. Recompute `quantum_nonce` from stored raw counts, job ID, and calibration snapshot.
2. Verify `record_hash` by recomputing over all fields (excluding `record_hash` itself).
3. Verify `raw_counts_hash` and `calibration_hash` by direct recomputation.
4. Confirm `schema_version == "witness-proofrecord-1.0"`.
5. Confirm `context_id` and `provider_instance` are present (non-empty).
6. Via IBM provider API: confirm job ID exists and provider-returned counts match stored counts.

Provenance note (preregistered): Provider verification confirms stored `job_id` and counts match the IBM provider API record at verification time. Does not confirm physical origin or quality of QPU randomness.

PASS criterion: All local recomputation checks pass AND provider job is found AND provider counts match. Provider API unavailability → GATE-STOP (not FAIL).

Expected outcome: PASS.

W2-C2 — Substitution / Tamper Detection (4 sub-trials)

Procedure: Four independent sub-trials, each altering exactly one field:

- (a) Alter `raw_counts` (swap two count values) → rerun nonce verification.
- (b) Alter `job_id` (append `_FORGED`) → rerun nonce and `record_hash` verification.
- (c) Alter `calibration_snapshot` (flip one readout_error value) → rerun nonce verification.
- (d) Alter `context_id` WITHOUT recomputing `record_hash` → rerun `record_hash` verification.

PASS criterion (each sub-trial): Substitution detected (`forgery_detected = True`).

Sub-trial	Altered field	Detection layer
a	<code>raw_counts</code>	<code>quantum_nonce</code>
b	<code>job_id</code>	<code>quantum_nonce</code> AND <code>record_hash</code>
c	<code>calibration_snapshot</code>	<code>quantum_nonce</code>
d	<code>context_id</code> (<code>record_hash</code> not updated)	<code>record_hash</code>

Overall PASS criterion: All 4 sub-trials detect the forgery.

Expected outcome: PASS (all 4 tamper attempts detected).

W2-C3 — Cross-Context Replay (2 sub-cases)

Procedure:

Sub-case (a): Alter `context_id` in the ProofRecord WITHOUT recomputing `record_hash`.

- Present to `verify_record_hash()`.

- Expected: `record_hash` mismatch → replay DENIED at field-integrity layer.

Sub-case (b): Alter `context_id` AND recompute `record_hash` with the new value.

- Present to `verify_record_hash()` — expected: `record_hash` is valid (attacker correctly recomputed it). This is the design boundary: `record_hash` provides field integrity only; it cannot enforce `context_id` binding.

- Present to ARK-457 5-dimensional context binding (`check_context_replay` from `ark457_replay.py`). Original context has `session=orig_ctx_id` ; replay context has `session=replay_ctx_id` . Expected: ARK-457 → DENY (session dimension mismatch).

Design boundary note (preregistered): Sub-case (b) `record_hash` being valid is the expected, correct, non-defect behaviour. The DENY verdict comes from ARK-457. This demonstrates the two-layer architecture: `record_hash` = field integrity; ARK-457 = context enforcement.

PASS criterion (each sub-case): Replay is DENIED (regardless of which layer fires).

Expected outcome (both): DENY (PASS).

8. Overall Verdict Rule

PASS iff all three cases (W2-C1, W2-C2, W2-C3) meet their stated criteria, and all sub-trials / sub-cases within each case meet their stated criteria.

Any single sub-trial / sub-case failing its criterion = overall FAIL, published as-is with full root-cause analysis. A follow-up record (WITNESS-2b) may be registered if warranted.

9. Related Works

- ARK Corpus (ExecutionProof series): concept DOI 10.5281/zenodo.21398675
 - ARK-457 (cross-context replay, most directly related): DOI 10.5281/zenodo.21421742 — replay-protection logic used here as library.
 - WITNESS-1 (predecessor): DOI 10.5281/zenodo.21424324
WITNESS concept series DOI: 10.5281/zenodo.21424323
 - WITNESS-2 is the second record in the WITNESS series, in the same `witness-testbeds` repository under `witness-2/` . It is an independent experiment, not a revision of WITNESS-1.
-

10. Mandatory Scope Disclaimer

“Results apply within the tested circuit model, backend, qubits, calibration, shot counts, software harnesses, and stated experimental conditions. This is an experimental evidence record, not a universal security proof or production certification.”

This disclaimer is reproduced verbatim in the README and Zenodo deposit description.

11. MANIFEST Lock Reference

`MANIFEST.sha256` covers this preregistration document and all files under `witness-2/src/`. It is committed before any QPU job is submitted. Post-lock code changes are prohibited. Any pre-execution harness defect triggers gate-stop protocol.

Remnant Fieldworks RF Process. Preregistration-first. No post-hoc changes.